

DRCP Technical Presentation - Speaker Notes

Here is your script for the Technical Presentation. It is designed for an Engineering Manager or Lead. It keeps the technical concepts clear, explains the architecture simply, and shows off your solid engineering decisions without using overly complicated jargon.

Slide 1: Title Slide

What to say:

"Hello everyone. Today, I'm going to walk you through the technical architecture of the Distributed Resilience Control Plane, or DRCP.

As we scale our systems, our biggest engineering risk isn't just a single service going down—it's that a single failure will drag the rest of the system down with it. I built DRCP to prevent exactly that: to stop cascading failures automatically, before human engineers even have to intervene."

Slide 2: Problem Statement

What to say:

"Let's start with the problem we are solving. In a microservices environment, services constantly talk to each other.

If Service A goes down, Service B might keep sending requests to it. Service B then runs out of resources waiting for a response and crashes too. Then Service C crashes. This is a cascading failure.

Currently, we rely on monitoring tools that alert an on-call engineer, who then has to wake up, figure out what's wrong, and manually shut off traffic. That process is too slow. Furthermore, we don't have a tamper-proof way to prove exactly when a service went down for our SLA contracts."

Slide 3: What DRCP Does (Solution Overview)

What to say:

"DRCP is a fully automated control plane that sits alongside our services. It does four things:

1. It constantly monitors real-time traffic data from all our services.
 2. It detects when a service breaches its predefined SLA—like if the error rate goes too high.
 3. It immediately reacts by telling our network proxies to cut off traffic to the failing service.
 4. Finally, it records a permanent, immutable log of the incident on the blockchain so we have a perfect audit trail."
-

Slide 4: System Architecture Diagram

What to say:

"Here is how the data moves through the system.

Our Dashboard UI talks to the API Gateway using standard REST APIs.

When services send their health data, the API Gateway immediately pushes that data into **Kafka**. We use Kafka because it allows us to handle massive spikes in traffic without crashing our own tools.

Our Worker service consumes those Kafka messages and stores them in **Redis** to calculate real-time error rates. If the error rate gets too high, the Worker tells **Open Policy Agent (OPA)** to check the rules.

If OPA says 'Yes, the rule is broken', it triggers two things: it tells **Envoy xDS** to trip the circuit breaker and stop the traffic, and it tells our Blockchain Anchor to save the incident."

Slide 5: Tech Stack

What to say:

"I chose a tech stack optimized for high concurrency and reliability.

The entire backend is written in **Go 1.25** because it is extremely fast and handles thousands of simultaneous connections perfectly.

For data, we use PostgreSQL for permanent storage and Redis for high-speed, temporary calculations. For messaging, we use Kafka.

To manage the rules, we use Open Policy Agent (OPA), and to manage the network traffic, we hook directly into Envoy Proxy. Everything is containerized in Docker so it is completely portable."

Slide 6: Microservice Architecture

What to say:

"To ensure DRCP doesn't have a single point of failure itself, I broke it into four separate microservices:

1. **The API Server:** Handles all web traffic and database management.
 2. **The Telemetry Worker:** The heavy lifter that crunches the numbers from Kafka.
 3. **The xDS Server:** This specifically talks to Envoy to push network configuration updates.
 4. **The Blockchain Anchor:** Handles the slow, heavy cryptographic work of talking to the Ethereum network."
-

Slide 7: Data Flow - Telemetry Pipeline

What to say:

"Let's trace a single piece of health data through the pipeline.

A service sends a ping saying 'I just had an error.' The API drops this into a Kafka topic.

The Worker picks it up and puts it into a Redis 'sliding window'. Every single second, the Worker looks at the last 60 seconds of data in Redis and asks: 'What is the error rate right now?'

If the error rate hits 50%, it alerts the system, logs the incident, and trips the circuit breaker."

Slide 8: Key Technical Components

What to say:

"There are a few complex pieces I want to highlight.

First, the **Redis Sliding Window**. I used Redis 'Sorted Sets'. By using timestamps as the score, I can instantly delete data older than 60 seconds and calculate the current error rate in milliseconds.

Second, **Envoy xDS**. Instead of manually restarting network routers, I used the 'go-control-plane' library. This allows DRCP to stream live network configurations to Envoy via gRPC, shutting off traffic instantly.

Third, the **Blockchain Anchor**, which uses standard EIP-1559 Ethereum transactions to embed the incident hashes permanently into the blockchain data field."

Slide 9: Design Patterns Used

What to say:

"I heavily relied on proven software design patterns to build this.

It uses an **Event-Driven Architecture** to decouple the fast API from the heavy math processing.

It uses the **Circuit Breaker Pattern** to protect weak services.

Most importantly, it uses **Graceful Degradation**. If Kafka or Redis completely crash, DRCP doesn't die. It automatically falls back to a slower, direct processing mode so we never lose our safety net."

Slide 10: Project Structure

What to say:

"The codebase is organized cleanly using standard Go project layouts.

The `cmd/` folder holds the main entry points for our 4 microservices.

The `internal/` folder holds all our core business logic.

The `pkg/` folder holds our shared tools, like database connection logic and logging, which can be reused by any service."

Slide 11: Challenges & Solutions

What to say:

"This project was not without challenges.

The biggest challenge was calculating error rates in real-time without slowing down the system. That's why I moved the math into Redis.

Another major challenge was making sure the system worked locally for other developers. Setting up Kafka and Postgres locally is hard, so I built a fallback mechanism where the app automatically uses a simple SQLite database if Postgres isn't available, allowing zero-config local development."

Slide 12: Key Learnings

What to say:

"Building DRCP taught me a lot about distributed systems. I learned how to properly decouple systems using message brokers like Kafka. I learned how to use gRPC to stream data to Envoy proxies, and I learned how to integrate traditional web systems with blockchain networks to create untamperable audit logs."

Slide 13: Live Demo & API

What to say:

"I have deployed a live version of the control plane. We have a fully documented REST API that allows you to register new services, create SLA contracts, and ingest telemetry data programmatically. You can view the live state of all of this on our web dashboard."

Slide 14: Thank You!

What to say:

"Thank you for your time. DRCP represents a major step forward in automating our platform's reliability. I'd love to take any technical questions you have about the implementation or dive into the live demo!"